6 APPENDICES

6.1 Appendix A – Dec Super App

1. Environment Variables: In each directory, create a .env file if it does not already exist. This file will hold your environment variables.
2. Docker Commands: To manage the Docker containers for the Dec Super App, we have used a Make file for ease of use
3. Docker Image: in the root directory of your Node.js application, create a file named Docker file with the following content:

6.2 Appendix B –Docker Configuration

The Docker file is used to create a Docker image for DEC SUPER APP . It includes the necessary instructions to set up the environment, install dependencies, and run the application.

Docker File for Admin Page

```
1 FROM node:18-alpine
2 WORKDIR /app
3 COPY package.json package-lock.json ./
4 RUN npm install
5 COPY . .
6 EXPOSE 5173
7 CMD ["npm", "run", "dev"]
```

Docker File for Api Page

```
1 FROM node:22.14.0
2 WORKDIR /
3 COPY package*.json ./
4 RUN npm install
```

```
5 COPY . .
6 EXPOSE 3000
7 CMD ["node", "index.js"]
```

Docker-Compose

```
services:

  eduadmin:
    container_name: eduadmin-dockerize_c

    image: eduadmin-dockerize_i
    build:
      context: ./EduAdmin
      dockerfile: Dockerfile
    ports:
      - "5173:5173"

    volumes:
      - ./EduAdmin:/app
      - /app/node_modules
    command: npm run dev
```

Mobile Apk Build

npm install -g eas-cli

eas login

eas build:configure

eas build --platform android

Environment Variables for Api

MongoDB Uri: http://localhost:27017/edusuperapp

Admin email: admin@admin.com

Admin password :10987126@NDi

Chapa Secret Key: “your secret key”

Google Auth Secret Key: “your google key”

Port:3000

Environment Variables For Admin

Backend host: ”http://localhost:3000”

Environment Variables For Edu Mobile app

Backend host:”http://localhost:3000”

6.3 Appendix C – Sample code For Authentication Middleware

```
const { User } = require("../Model/User");
const { verifyAuthToken } = require("../utils/jwt");


module.exports = async (req, res, next) => {
  const token = req.headers["authorization"];
```

```
 if (!token) return res.status(401). send("Access denied. No token provided.");
 try {
  const decoded = verifyAuthToken(token);

  const users = await User.findOne({ auth: decoded._id });
  if (!users) return res.status(401).send("Invalid  token");

  req.user = decoded;

  next();
 } catch (err) {
  console.log(err);
  res.status(401).send("Invalid token");
 }
};
```